# CS336 assignment 2

## 1. Problem (benchmarking_script):

(b) Using a vocab size of 10000 and batch size of 4:

| mode | model_size | seq_len | warmup_steps | measure_steps | mean_ms | std_ms |
|---|---|---|---|---|---|---|
| forward | small | 128 | 5 | 10 | 17.3085 | 0.161352 |
| train | small | 128 | 5 | 10 | 47.4959 | 0.711806 |

(c) Without the warm-up steps:

| mode | model_size | seq_len | warmup_steps | measure_steps | mean_ms | std_ms |
|---|---|---|---|---|---|---|
| forward | small | 128 | 0 | 10 | 91.4496 | 234.847 |
| train | small | 128 | 0 | 10 | 151.595 | 328.019 |

With 1 warm up step:

| mode | model_size | seq_len | warmup_steps | measure_steps | mean_ms | std_ms |
|---|---|---|---|---|---|---|
| forward | small | 128 | 1 | 10 | 17.2387 | 0.21551 |
| train | small | 128 | 1 | 10 | 47.3618 | 0.375036 |

## Problem(nsys_profile):

(a) Forward pass: ~16 ms

(b) elementwise kernel?

## Problem(mixed_precision_accumulation):

For the first one:

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s+= torch.tensor(0.01,dtype=torch.float32)
print(s)
```

The result is : `tensor(10.0001)`

For the second one:

```
s = torch.tensor(0,dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)
```

The result is `tensor(9.9531, dtype=torch.float16)`

For the third one:

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)
```

The result is `tensor(10.0021)`

For the last one:

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01,dtype=torch.float16)
    s += x.type(torch.float32)
print(s)
```

The result is: `tensor(10.0021)`

The float32 accumulation is fairly accurate: summing `0.01` a thousand times gives `10.0001`, which is very close to the exact value `10`. In contrast, pure float16 accumulation is much less accurate (`9.9531`) because float16 has much lower precision, so rounding error builds up significantly during repeated additions. When the accumulator is kept in float32 but the added value is first represented in float16, the result becomes `10.0021`; **this shows that mixed-precision accumulation avoids the large error of float16 summation, but it still inherits the quantization error from storing `0.01` in float16 before accumulation.**

# Problem (benchmarking_mixed_precision):

(a)

- model parameters: `FP32`
- output of the first feed-forward layer: `FP16`
- `LayerNorm`: `FP32`, for numerical stability, since this operator is sensitive to numerical errors.
- `logits`: Usually `FP16` for prediction, `FP32` for backward pass
- loss: `FP32`
- gradients: `FP32`

(b) Layer normalization is sensitive to mixed precision because it involves computing the mean and variance, subtracting nearly equal numbers, and dividing by a small standard deviation; these steps can amplify rounding error and lead to unstable results in FP16. BF16 usually reduces the need for special handling compared with FP16 because it has the same exponent range as FP32 and is therefore much less prone to overflow or underflow, but it still has limited mantissa precision, so keeping layer normalization in FP32 is often still preferred for numerical stability.
(c) Small model with / without mixed precision:

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | small | 128 | 5 | 10 | True | 12.7635 | 0.749513 |

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | small | 128 | 5 | 10 | False | 15.8811 | 0.156729 |

Medium model with/without mixed precision:

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | medium | 128 | 5 | 10 | True | 26.3477 | 0.652189 |

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | medium | 128 | 5 | 10 | False | 34.998 | 0.132152 |

Large model with/without mixed precision:

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | large | 128 | 5 | 10 | True | 40.908 | 2.22375 |

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | large | 128 | 5 | 10 | False | 77.3654 | 2.60084 |

extra large model with/without mixed precision:

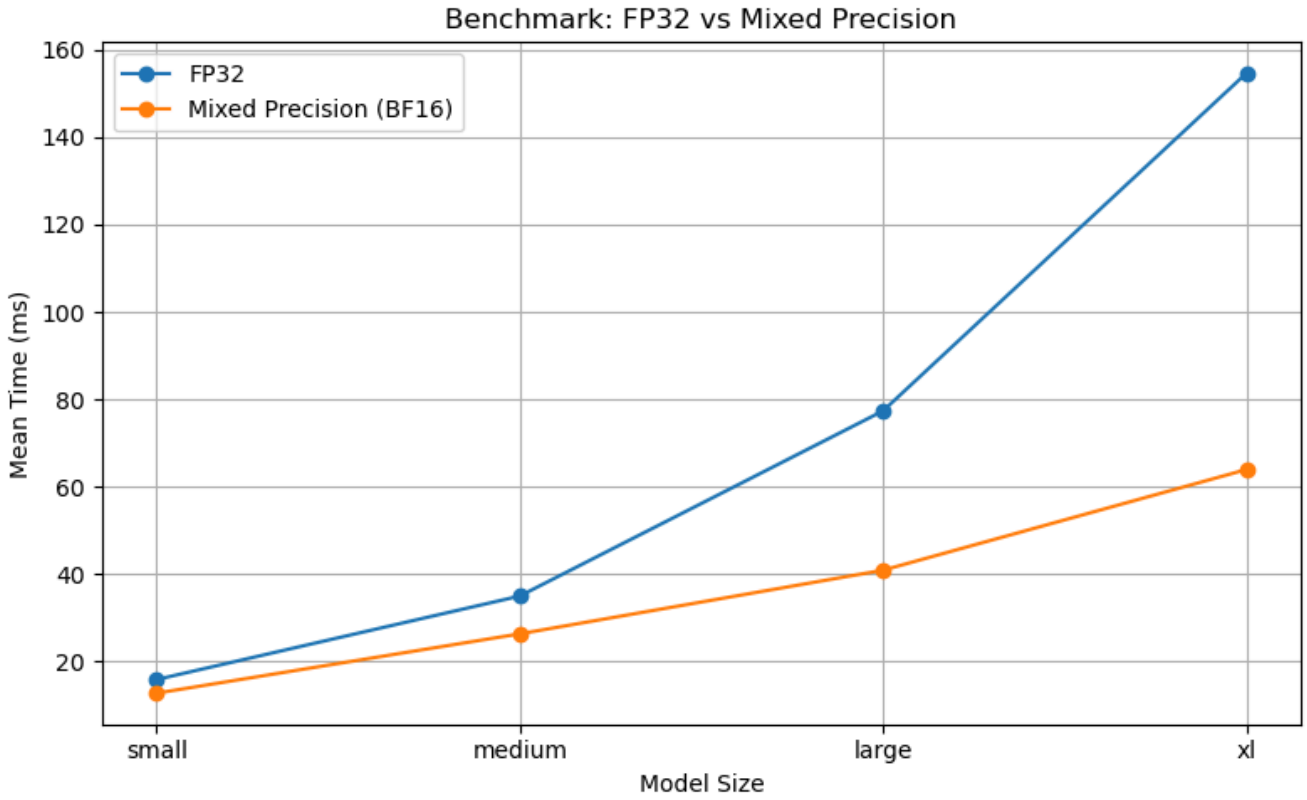
| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | xl | 128 | 5 | 10 | True | 63.9334 | 0.308356 |

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | xl | 128 | 5 | 10 | False | 154.61 | 2.92403 |

2.7B model with/without mixed precision:

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | 2.7b | 128 | 5 | 10 | True | 5270.58 | 229.145 |

| mode | model_size | seq_len | warmup_steps | measure_steps | mixed_precision | mean_ms | std_ms |
|---|---|---|---|---|---|---|---|
| forward | 2.7b | 128 | 5 | 10 | False | 238.868 | 6.52695 |




# Problem(memory_profiling):

(b) peak memory usage of 128 context length when doing a forward pass:

| task | mode | model_size | seq_len | mixed_precision | peak_allocated_mb | peak_reserved_mb | snapshot_path |
|---|---|---|---|---|---|---|---|
| memory_profile | forward | 2.7b | 128 | False | 13212.8 | 13246 | snapshots/memory_128_profile.pickle |

(c) when using mixed-precision

| task | mode | model_size | seq_len | mixed_precision | peak_allocated_mb | peak_reserved_mb | snapshot_path |
|---|---|---|---|---|---|---|---|
| memory_profile | forward | 2.7b | 128 | True | 19599.1 | 19838 | snapshots/memory_128_profile.pickle |

# Problem(pytorch_attention):

Final results:

| d_model | seq_len | forward_mean_ms | forward_std_ms | forward_memory | forward_peak_mem | backward_mean_ms | backward_std_ms | status | OOM_stage |
|---|---|---|---|---|---|---|---|---|---|
| 16 | 256 | 0.0554917 | 0.0230466 | 10.6255 | 12.625 | 0.363668 | 0.0616856 | ok | |
| 16 | 1024 | 0.256673 | 0.0229716 | 50.2505 | 82.25 | 0.844411 | 0.0291218 | ok | |
| 16 | 4096 | 5.54749 | 0.326864 | 536.25 | 1048.25 | 16.5557 | 1.09492 | ok | |
| 16 | 8192 | 24.2796 | 1.86975 | 2080.25 | 4128.25 | 72.5202 | 5.29148 | ok | |
| 32 | 256 | 0.136408 | 0.0967467 | 19.2505 | 21.25 | 0.586932 | 0.153988 | ok | |
| 32 | 1024 | 0.377658 | 0.11324 | 52.2505 | 84.25 | 1.37831 | 1.45869 | ok | |
| 32 | 4096 | 6.72134 | 0.528951 | 544.25 | 1056.25 | 17.3003 | 1.06322 | ok | |
| 32 | 8192 | 26.0487 | 3.33289 | 2096.25 | 4144.25 | 71.8574 | 9.69883 | ok | |
| 64 | 256 | 0.0665244 | 0.0129357 | 20.2505 | 22.25 | 0.410466 | 0.0946627 | ok | |
| 64 | 1024 | 0.327476 | 0.0280983 | 56.2505 | 88.25 | 1.058 | 0.0358889 | ok | |
| 64 | 4096 | 6.72437 | 1.48504 | 560.25 | 1072.25 | 23.1956 | 4.5061 | ok | |
| 64 | 8192 | 27.6351 | 4.43799 | 2128.25 | 4176.25 | 72.1703 | 3.71965 | ok | |
| 128 | 256 | 0.0968295 | 0.0167991 | 22.2505 | 24.25 | 0.439063 | 0.0581576 | ok | |
| 128 | 1024 | 0.511575 | 0.0328726 | 64.2505 | 96.25 | 1.52776 | 0.0152369 | ok | |
| 128 | 4096 | 9.24979 | 0.338306 | 592.25 | 1104.25 | 26.3233 | 0.999511 | ok | |
| 128 | 8192 | 44.1919 | 6.99523 | 2192.25 | 4240.25 | 108.084 | 7.29877 | ok | |

Not capable when seq_len=16384.

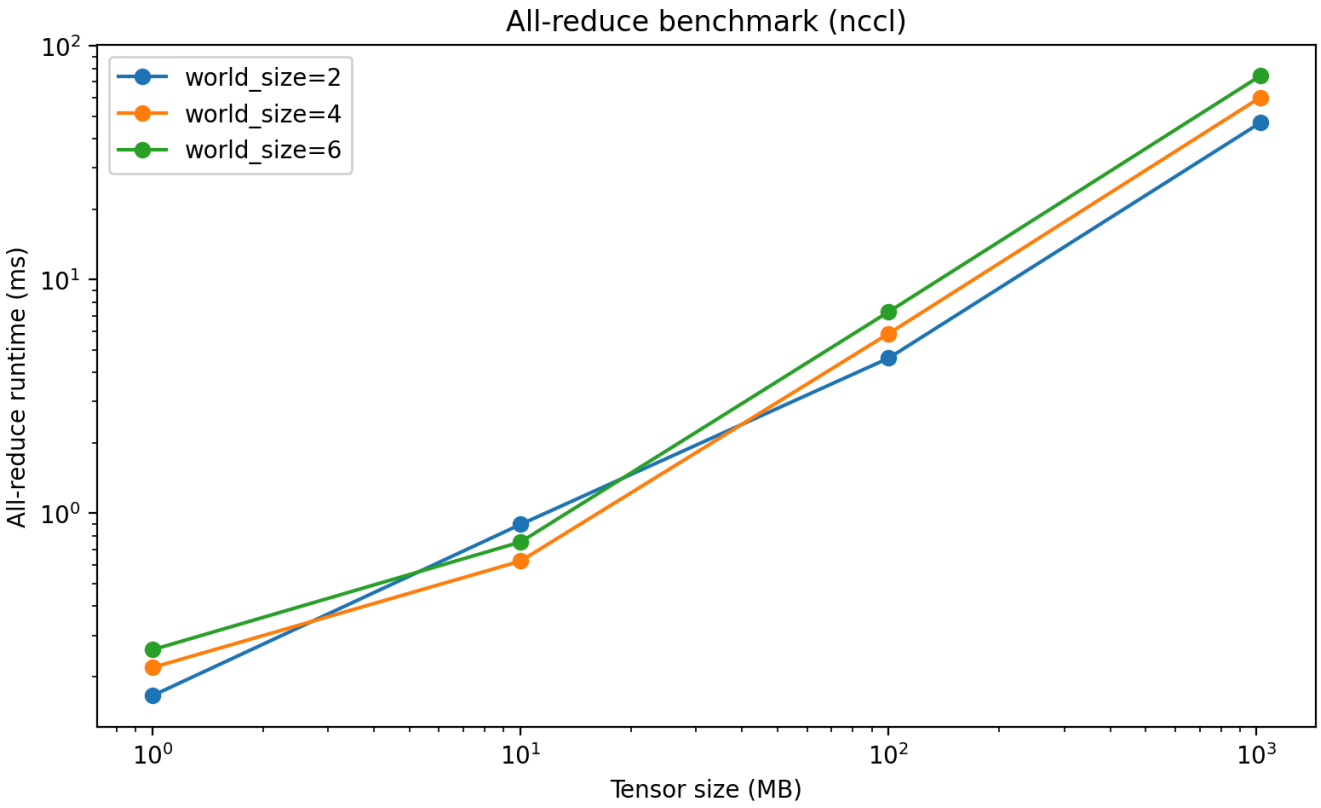
# Problem (torch_compile):Problem (torch_compile):

(a) Final results:

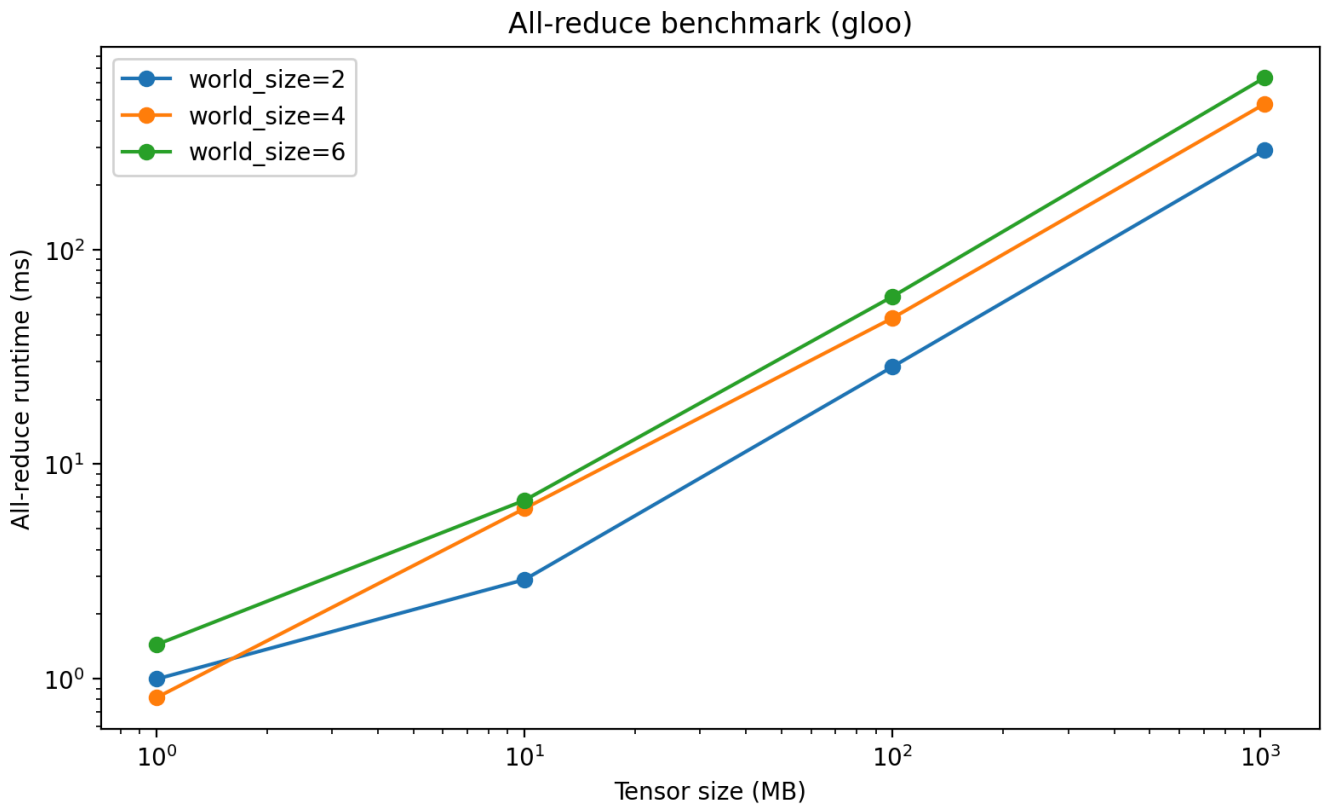
| d_model | seq_len | forward_mean_ms | forward_std_ms | forward_memory | forward_peak_mem | backward_mean_ms | backward_std_ms | status | OOM_stage | variant |
|---|---|---|---|---|---|---|---|---|---|---|
| 16 | 256 | 0.0595534 | 0.0214584 | 10.6255 | 12.625 | 0.362004 | 0.0325624 | ok | | eager |
| 16 | 256 | 0.0666949 | 0.00792504 | 18.7505 | 18.7505 | 0.422598 | 0.065273 | ok | | compiled |
| 16 | 1024 | 1.58189 | 2.92027 | 50.2505 | 82.25 | 0.906708 | 0.02183 | ok | | eager |
| 16 | 1024 | 0.272451 | 0.0123977 | 51.2505 | 66.25 | 0.749012 | 0.040449 | ok | | compiled |
| 16 | 4096 | 5.95533 | 0.17443 | 536.25 | 1048.25 | 16.7974 | 0.422041 | ok | | eager |
| 16 | 4096 | 4.01471 | 0.189783 | 540.25 | 540.251 | 10.7426 | 0.110462 | ok | | compiled |
| 16 | 8192 | 23.1602 | 0.503194 | 2080.25 | 4128.25 | 65.556 | 2.59591 | ok | | eager |
| 16 | 8192 | 16.2087 | 0.317897 | 2088.25 | 2088.25 | 43.7239 | 0.96107 | ok | | compiled |
| 32 | 256 | 0.0660082 | 0.0314828 | 19.2505 | 21.25 | 0.365015 | 0.0492552 | ok | | eager |
| 32 | 256 | 0.128048 | 0.0107736 | 19.2505 | 19.2505 | 0.459304 | 0.0368328 | ok | | compiled |
| 32 | 1024 | 0.303693 | 0.025017 | 52.2505 | 84.25 | 1.01092 | 0.0157347 | ok | | eager |
| 32 | 1024 | 0.402383 | 0.0329252 | 52.2505 | 52.2515 | 0.876309 | 0.0385342 | ok | | compiled |
| 32 | 4096 | 5.89407 | 0.175438 | 544.25 | 1056.25 | 16.9313 | 0.14786 | ok | | eager |
| 32 | 4096 | 4.48549 | 0.585957 | 544.25 | 544.251 | 11.8216 | 0.167968 | ok | | compiled |
| 32 | 8192 | 23.3611 | 0.346321 | 2096.25 | 4144.25 | 66.112 | 2.67215 | ok | | eager |
| 32 | 8192 | 20.459 | 0.293301 | 2096.25 | 2096.25 | 54.6682 | 0.912517 | ok | | compiled |
| 64 | 256 | 0.0787964 | 0.0276778 | 20.2505 | 22.25 | 0.373677 | 0.0314602 | ok | | eager |
| 64 | 256 | 0.118972 | 0.0112757 | 20.2505 | 20.2515 | 0.434062 | 0.0410511 | ok | | compiled |
| 64 | 1024 | 0.306156 | 0.0206039 | 56.2505 | 88.25 | 1.02087 | 0.0149286 | ok | | eager |
| 64 | 1024 | 0.457113 | 0.0374611 | 56.2505 | 56.2515 | 0.970468 | 0.0182861 | ok | | compiled |
| 64 | 4096 | 6.31662 | 0.179656 | 560.25 | 1072.25 | 18.1127 | 0.146938 | ok | | eager |
| 64 | 4096 | 4.84234 | 0.116229 | 560.25 | 560.252 | 13.3327 | 0.441557 | ok | | compiled |
| 64 | 8192 | 25.6012 | 0.503344 | 2128.25 | 4176.25 | 71.7106 | 3.34089 | ok | | eager |
| 64 | 8192 | 22.0478 | 0.254851 | 2128.25 | 2128.25 | 59.4284 | 0.690121 | ok | | compiled |
| 128 | 256 | 0.0945996 | 0.00345708 | 22.2505 | 24.25 | 0.355576 | 0.0379118 | ok | | eager |
| 128 | 256 | 0.136937 | 0.0149482 | 22.2505 | 22.2515 | 0.451685 | 0.0207354 | ok | | compiled |
| 128 | 1024 | 0.531924 | 0.0354225 | 64.2505 | 96.25 | 1.53896 | 0.0111809 | ok | | eager |
| 128 | 1024 | 0.633676 | 0.046523 | 64.2505 | 64.2515 | 1.53196 | 0.0169867 | ok | | compiled |
| 128 | 4096 | 9.14615 | 0.0958716 | 592.25 | 1104.25 | 25.901 | 0.284455 | ok | | eager |
| 128 | 4096 | 7.94098 | 0.235088 | 592.25 | 592.252 | 21.7356 | 0.682979 | ok | | compiled |
| 128 | 8192 | 36.7357 | 0.496851 | 2192.25 | 4240.25 | 104.779 | 2.69659 | ok | | eager |
| 128 | 8192 | 33.9599 | 0.754328 | 2192.25 | 2192.25 | 92.0186 | 1.12481 | ok | | compiled |

## Problem (distributed_communication_single_node)

| backend | world_size | size_mb | global_mean_ms |
|---|---|---|---|
| nccl | 2 | 1 | 0.165847 |
| nccl | 2 | 10 | 0.895438 |
| nccl | 2 | 100 | 4.60458 |
| nccl | 2 | 1024 | 46.8356 |
| nccl | 4 | 1 | 0.218642 |
| nccl | 4 | 10 | 0.623951 |
| nccl | 4 | 100 | 5.86472 |
| nccl | 4 | 1024 | 60.1007 |
| nccl | 6 | 1 | 0.260486 |
| nccl | 6 | 10 | 0.752872 |
| nccl | 6 | 100 | 7.26884 |
| nccl | 6 | 1024 | 74.3396 |



| backend | world_size | size_mb | global_mean_ms |
|---|---|---|---|
| gloo | 2 | 1 | 0.995526 |
| gloo | 2 | 10 | 2.89596 |
| gloo | 2 | 100 | 28.4776 |
| gloo | 2 | 1024 | 291.577 |
| gloo | 4 | 1 | 0.816098 |
| gloo | 4 | 10 | 6.21365 |
| gloo | 4 | 100 | 48.0423 |
| gloo | 4 | 1024 | 478.17 |
| gloo | 6 | 1 | 1.44101 |
| gloo | 6 | 10 | 6.78617 |
| gloo | 6 | 100 | 60.5566 |
| gloo | 6 | 1024 | 633.586 |




## Problem (naive_ddp_benchmarking)

Per-rank results:
rank=0, avg_step_time_ms=392.118, avg_comm_prop=0.463
rank=1, avg_step_time_ms=391.937, avg_comm_prop=0.468

Mean step time across ranks: 392.028 ms
Mean communication proportion across ranks: 0.465

# Problem (minimal_ddp_flat_benchmarking)

Per-rank results:
rank=0, avg_step_time_ms=442.561, avg_comm_prop=0.436
rank=1, avg_step_time_ms=442.877, avg_comm_prop=0.466

Mean step time across ranks: 442.719 ms
Mean communication proportion across ranks: 0.451

# Problem(ddp_overlap_individual_parameters_benchmarking)

Per-rank results:
rank=0, avg_step_time_ms=360.420, avg_finish_wait_ms=5.756, avg_finish_wait_prop=0.016
rank=1, avg_step_time_ms=360.521, avg_finish_wait_ms=5.597, avg_finish_wait_prop=0.016

Mean step time across ranks: 360.470 ms
Mean finish-wait time across ranks: 5.676 ms
Mean finish-wait proportion across ranks: 0.016

# Problem(ddp_bucketed_benchmarking)

(b)

The size of each bucket is:

$$b = \frac{s}{n_b}$$

After the gradients of the last bucket is computed, the overhead for communication is :

$$\frac{s}{n_b w}$$

For each bucket, overhead associated with communication call is $o$

Therefore, the overall DDP overhead is:

$$T = n_b o + \frac{s}{n_b w}$$

Therefore,

$$argmin_{n_b} T = \sqrt{\frac{s}{wo}}$$

# Problem (communication_accounting)

(a):

Model weights: 4 Bytes / Parameter

Accumulated Grads: 4 Bytes / Parameter

Optimizer States: 8 Bytes / Parameter

Total: 16 Bytes / Parameter

Total Parameters: $2 \cdot 16384 \cdot 53248 \cdot 126 = 219848638464$

Total Memory:

$$\frac{219848638464 \cdot 16}{1024^3} = 3276GB$$

If use BF16 for backward:

$$\frac{219848638464 \cdot 2}{1024^3} = 409.5GB$$

Therefore saving $409.5$ GB memories.

Total memory requires $ceil(3276/80) = 41$ H100 GPUs.

(b):

First, for parameters: Each GPU takes :

$$\frac{3276}{N_{FSDP}}GB$$

of parameters.

Then, consider activations. One half of the activations are stored locally, while the other half activations are shared.

Each block should save $BS(d_{model} + d_{ff})$ number of activations, totally (in GB):

$$\frac{BS \cdot 126 \cdot (16384 + 53248) \cdot 2}{1024^3} = 0.016BS$$

Define:

$$A = 0.016BS$$

Thus, for each device:

$$Total = \frac{3276}{N_{FSDP}} + \frac{A}{2} + \frac{A}{2 \cdot N_{FSDP}}$$

Lastily,

$$N_{FSDP} > \frac{3276 + 0.00817(BS)}{95 + 0.00817(BS)}$$